

```
package Guild;

public abstract class Miembro {
    private static final int MAX_ENERGIA = 100;
    private static int numMiembros = 0;
    private final String identificador;
    private int nivel;
    private int energiaActual;
    private int misionesCompletadas;

    public Miembro(String identificador, int nivel, int
energiaActual) {
        if (identificador == null || identificador.isBlank()) {
            throw new IllegalArgumentException("El identificador no
puede estar vacío");
        }
        if (nivel < 1 || nivel > 50) {
            throw new IllegalArgumentException("El nivel debe estar
entre 1 y 50");
        }
        if (energiaActual < 0 || energiaActual > MAX_ENERGIA) {
            throw new IllegalArgumentException("La energía debe
estar entre 0 y " + MAX_ENERGIA);
        }

        this.identificador = identificador;
        this.nivel = nivel;
        this.energiaActual = energiaActual;
        this.misionesCompletadas = 0;
        numMiembros++;
    }

    public void recuperarEnergia(int cantidad) {
        if (cantidad < 0) {
            throw new IllegalArgumentException("La cantidad a
recuperar no puede ser negativa");
        }
        if ((energiaActual + cantidad) > MAX_ENERGIA) {
            energiaActual = MAX_ENERGIA;
        } else {
            energiaActual += cantidad;
        }
    }

    public void gastarEnergia(int cantidad) {
        if (cantidad < 0) {
            throw new IllegalArgumentException("La cantidad a
gastar no puede ser negativa");
        }
    }
}
```

```

    }
    if (energiaActual >= cantidad) {
        energiaActual -= cantidad;
    } else {
        throw new RuntimeException("Energia insuficiente");
    }
}

@Override
public String toString() {
    return "Miembro{" +
        "identificador=" + identificador + '\'' +
        ", nivel=" + nivel +
        ", energiaActual=" + energiaActual +
        ", misionesCompletadas=" + misionesCompletadas +
        '\'';
}

public void completarMision() {
    misionesCompletadas++;
}

public void participarMision(int costeBase) {
    int costeReal = calcularGastoEnergia(costeBase);
    int ayuda = calcularAportacionEspecial();
    gastarEnergia(costeReal - ayuda);
    completarMision();
}

public abstract int calcularGastoEnergia(int costeBase);
public abstract int calcularAportacionEspecial();

public static int getNumMiembros() {
    return numMiembros;
}

public String getIdentificador() {
    return identificador;
}

public int getNivel() {
    return nivel;
}

public int getEnergiaActual() {
    return energiaActual;
}

```

```

        public int getMisionesCompletadas() {
            return misionesCompletadas;
        }
    }

package Guild;

class Guerrero extends Miembro {
    private int fuerza;

    public Guerrero(String identificador, int nivel, int
energiaActual, int fuerza) {
        super(identificador, nivel, energiaActual);
        if (fuerza < 1 || fuerza > 100) {
            throw new IllegalArgumentException("La fuerza tiene que
estar entre 1 y 100");
        }
        this.fuerza = fuerza;
    }

    @Override
    public int calcularGastoEnergia(int costeBase) {
        return costeBase + 5;
    }

    @Override
    public int calcularAportacionEspecial() {
        return fuerza / 2;
    }
}

```

```

package Guild;

class Maga extends Miembro {
    private int mana;

    public Maga(String identificador, int nivel, int energiaActual,
int mana) {
        super(identificador, nivel, energiaActual);
        if (mana < 1 || mana > 100) {
            throw new IllegalArgumentException("La mana tiene que
estar entre 1 y 100");
        }
        this.mana = mana;
    }

    @Override
    public int calcularGastoEnergia(int costeBase) {

```

```

        return costeBase - 3;
    }

    @Override
    public int calcularAportacionEspecial() {
        return mana;
    }
}

package Guild;

class Sanador extends Miembro {
    private int curacion;

    public Sanador(String identificador, int nivel, int
energiaActual, int curacion) {
        super(identificador, nivel, energiaActual);
        if (curacion < 1 || curacion > 100) {
            throw new IllegalArgumentException("La curación tiene
que estar entre 1 y 100");
        }
        this.curacion = curacion;
    }

    @Override
    public int calcularGastoEnergia(int costeBase) {
        return costeBase;
    }

    @Override
    public int calcularAportacionEspecial() {
        return curacion * 2;
    }
}

package Guild;

public interface ReductorGasto {
    int reducirGasto(int costeOriginal);
}

package Guild;

public interface Curador {
    void curar(Miembro miembro);
}

package Guild;

```

```

public enum Dificultad {
    FACIL("Fácil", 1.0, 10),
    MEDIA("Media", 1.5, 20),
    DIFICIL("Difícil", 2.0, 30),
    EPICA("Épica", 3.0, 40);

    private final String nombreVisible;
    private final double multiplicador;
    private final int energiaMinima;

    private Dificultad(String nombreVisible, double multiplicador,
int energiaMinima) {
        this.nombreVisible = nombreVisible;
        this.multiplicador = multiplicador;
        this.energiaMinima = energiaMinima;
    }

    public int calcularCosteEnergia() {
        return (int) (energiaMinima * multiplicador);
    }

    public String getNombreVisible() {
        return nombreVisible;
    }
}

package Guild;

public class Misión {
    private String codigo;
    private String nombre;
    private Dificultad dificultad;
    private boolean completada;

    public Misión(String codigo, String nombre, Dificultad
dificultad) {
        if (codigo == null || codigo.isBlank()) {
            throw new IllegalArgumentException("El código no puede
estar vacío");
        }
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede
estar vacío");
        }
        if (dificultad == null) {
            throw new IllegalArgumentException("La dificultad no
puede ser null");

```

```

    }

    this.codigo = codigo;
    this.nombre = nombre;
    this.dificultad = dificultad;
    this.completada = false;
}

public void completarMision(Miembro miembro) {
    if (completada) {
        throw new MisionNoDisponibleException("La mision " +
codigo + " ya estaba completada");
    }

    int coste = dificultad.calcularCosteEnergia();

    if (miembro instanceof ReductorGasto) {
        coste = ((ReductorGasto) miembro).reducirGasto(coste);
    }

    miembro.participarMision(coste);

    if (miembro instanceof Curador) {
        ((Curador) miembro).curar(miembro);
    }

    this.completada = true;
}

public String descripcionBreve() {
    return "Mision " + codigo + ": " + nombre + " (" +
dificultad + ")";
}

@Override
public String toString() {
    return descripcionBreve();
}

public String getCodigo() {
    return codigo;
}

public Dificultad getDificultad() {
    return dificultad;
}
}

```

```
package Guild;

public class Equipo {
    private String nombre;
    private Miembro lider;
    private Miembro[] miembros;
    private int contador;

    public Equipo(String nombre, Miembro lider) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede
estar vacio");
        }
        if (lider == null) {
            throw new IllegalArgumentException("El lider no puede
ser null");
        }

        this.nombre = nombre;
        this.lider = lider;
        this.miembros = new Miembro[5];
        this.miembros[0] = lider;
        this.contador = 1;
    }

    public void agregarMiembro(Miembro m) {
        if (m == null) {
            throw new IllegalArgumentException("El miembro no puede
ser null");
        }
        if (contador >= 5) {
            throw new IllegalStateException("No se pueden añadir
mas miembros, limite 5");
        }

        for (int i = 0; i < contador; i++) {
            if
(miembros[i].getIdentificador().equals(m.getIdentificador())) {
                throw new IllegalArgumentException("Ya existe un
miembro con ese ID");
            }
        }

        miembros[contador] = m;
        contador++;
    }

    public void eliminarMiembro(String id) {
```

```

        if (id.equals(lider.getIdentificador())) {
            throw new IllegalStateException("No se puede eliminar
al lider");
        }

        for (int i = 0; i < contador; i++) {
            if (miembros[i].getIdentificador().equals(id)) {
                miembros[i] = miembros[contador - 1];
                miembros[contador - 1] = null;
                contador--;
                return;
            }
        }
        throw new IllegalArgumentException("No se encontro miembro
con ID: " + id);
    }

    public int energiaTotal() {
        int total = 0;
        for (int i = 0; i < contador; i++) {
            total += miembros[i].getEnergiaActual();
        }
        return total;
    }

    public void listar() {
        System.out.println("Equipo: " + nombre + " (lider: " +
lider.getIdentificador() + ")");
        for (int i = 0; i < contador; i++) {
            System.out.println("  - " + miembros[i]);
        }
        System.out.println("Energia total: " + energiaTotal());
    }
}

```

```

package Guild;

```

```

public class EnergiaInsuficienteException extends Exception {
    public EnergiaInsuficienteException(String mensaje) {
        super(mensaje);
    }
}

```

```

package Guild;

```

```

public class MisionNoDisponibleException extends RuntimeException
{
    public MisionNoDisponibleException(String mensaje) {

```



```

        super(mensaje);
    }
}

package Guild;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== GUILDHUB ===\n");

        Guerrero g = new Guerrero("G001", 10, 100, 80);
        Maga m = new Maga("M001", 8, 90, 75);
        Sanador s = new Sanador("S001", 12, 95, 50);

        System.out.println("Miembros creados:");
        System.out.println(g);
        System.out.println(m);
        System.out.println(s);
        System.out.println("Total miembros: " +
Miembro.getNumMiembros() + "\n");

        Mision mision = new Mision("MSN01", "Matar dragon",
Dificultad.EPICA);
        System.out.println("Mision: " + mision);
        System.out.println("Coste: " +
mision.getDificultad().calcularCosteEnergia() + "\n");

        Equipo e = new Equipo("Los mejores", g);
        e.agregarMiembro(m);
        e.agregarMiembro(s);
        System.out.println("Equipo creado:");
        e.listar();
        System.out.println();

        System.out.println("Participando en mision...");
        mision.completarMision(g);
        System.out.println("Despues de mision: " + g + "\n");

        System.out.println("Probando caso incorrecto...");
        try {
            Guerrero debil = new Guerrero("G002", 5, 10, 50);
            Mision dificil = new Mision("MSN02", "Imposible",
Dificultad.EPICA);
            System.out.println("Guerrero debil: " + debil);
            dificil.completarMision(debil);
        } catch (Exception ex) {
            System.out.println("Error: " + ex.getMessage() + "\n");
        }
    }
}

```

```

        System.out.println("Polimorfismo:");
        Miembro[] lista = {g, m, s};
        for (Miembro x : lista) {
            System.out.println(x + " aporta: " +
x.calcularAportacionEspecial());
        }
        System.out.println();

        System.out.println("Coleccion de misiones:");
        Mision[] misiones = {
            new Mision("M1", "Facil", Dificultad.FACIL),
            new Mision("M2", "Media", Dificultad.MEDIA),
            mision
        };
        for (Mision x : misiones) {
            System.out.println(" " + x);
        }
        System.out.println();

        System.out.println("Costes por dificultad:");
        for (Dificultad d : Dificultad.values()) {
            System.out.println(" " + d + ": " +
d.calcularCosteEnergia());
        }
        System.out.println();

        System.out.println("Capacidades especiales:");
        if (g instanceof ReductorGasto) {
            System.out.println(" Guerrero reduce gasto");
        }
        if (s instanceof Curador) {
            System.out.println(" Sanador puede curar");
        }
        System.out.println();

        System.out.println("Estado final:");
        System.out.println(g);
        System.out.println(m);
        System.out.println(s);
        e.listar();
    }
}

```